Haklısınız, bu yaklaşım yazılım ekosisteminin **sürdürülebilirliği, çeşitliliği ve güvenliği** açısından çok daha güçlü bir temel sunuyor. Doğrudan işletim sistemine (OS) yapay zeka entegre etmek (Windows Copilot gibi) son kullanıcı için pratik bir "hazır tüketim" sağlasa da, geliştiricilere yönelik **framework ve kütüphaneler sunmak** çok daha geniş kapsamlı bir inovasyon dalgası yaratır.

Bu iki yaklaşımın doğurduğu farkları ve sizin belirttiğiniz yöntemin neden daha avantajlı olabileceğini birkaç temel başlıkta inceleyebiliriz:

1. Tekelleşme Karşıtlığı ve Çeşitlilik

- **OS Entegrasyonu:** İşletim sistemi seviyesindeki bir yapay zeka, kullanıcıyı tek bir modele veya ekosisteme (örneğin Microsoft-OpenAI ortaklığına) mahkum eder. Rekabeti azaltır.
- **Framework Yaklaşımı:** Yazılımcılara araçlar vermek, binlerce farklı fikrin ve niş uygulamanın doğmasını sağlar. Sağlık, finans, eğitim gibi spesifik alanlar için optimize edilmiş, yerel (local) veya açık kaynaklı yüzlerce farklı yapay zeka servisi üretilebilir.

2. Güvenlik ve Gizlilik (Privacy)

- **OS Entegrasyonu:** İşletim sisteminin her hareketinizi izlemesi (örneğin Windows'un tartışmalı *Recall* özelliği gibi) ciddi gizlilik endişeleri doğurur. Kurumsal firmalar verilerinin işletim sistemi seviyesinde işlenmesini istemez.
- **Framework Yaklaşımı:** Giral geliştiriciler, kütüphaneleri kullanarak verinin tamamen cihaz içinde kaldığı (**on-device AI**), internete çıkmayan, güvenli ve regülasyonlara uygun (GDPR, KVKK) kapalı devre sistemler inşa edebilirler.

3. Kaynak Yönetimi ve Performans

- **OS Entegrasyonu:** İşletim sisteminin arkasında sürekli çalışan devasa bir yapay zeka katmanı, sistem kaynaklarını (RAM, CPU, NPU) arka planda gereksiz yere tüketebilir.
- **Framework Yaklaşımı:** Kullanıcı yapay zekayı sadece o kütüphaneyle yazılmış spesifik bir uygulamayı açtığında çalıştırır. Uygulama kapandığında kaynaklar serbest kalır.

4. Yazılımcıları Teşvik Etmek ve Ekonomik Döngü

- **OS Entegrasyonu:** Büyük teknoloji şirketi tüm pastayı kendisi yer. Üçüncü parti yazılımcıların işletim sistemi için uygulama geliştirme motivasyonunu kırar.
- **Framework Yaklaşımı:** Apple'ın **CoreML** veya Google'ın **MediaPipe** / **TensorFlow Lite** ile yaptığı gibi, işletim sistemi donanımı (NPU/GPU) API'ler aracılığıyla yazılımcıya açılır. Bu da yeni bir "yapay zeka uygulama ekonomisi" yaratır.

Sektör Aslında İkisini Bir Arada Yapıyor

Bugün Apple, Microsoft ve Google gibi devler tam olarak sizin söylediğiniz yolu da yürütüyorlar. Bir yandan son kullanıcıya hitap eden yapay zeka özellikleri (Apple Intelligence, Windows Copilot) sunarken, diğer yandan geliştiriciler için **Windows Copilot Library**, **Core ML**, **ONNX Runtime** gibi kütüphaneleri sürekli güncelliyorlar. Çünkü ekosistemi elinde tutmanın yolunun

yazılımcıları arkana almaktan geçtiğini biliyorlar.

Yazılımcıların kendi uygulamalarında yapay zekayı esnekçe kullanabilmesi vizyonunuz oldukça isabetli. Eğer bu alanda bir proje geliştirmeyi düşünüyorsanız;

- Bu framework'ün **hangi programlama dilini** (Python, Rust, C++ vb.) hedeflemesini isterdiniz?
- Hangi **spesifik problemi** (örneğin cihaz içi metin işleme, ses tanıma veya veri analizi) çözmeye odaklanırdınız?